Lab experiment 1 :

```
C:\Windows\System32\cmd.e

25-01-2026  18:37               311 labexp1.l
25-01-2026  18:43               288 labexp2.l
25-01-2026  18:59               302 labexp3.l
25-01-2026  19:07               329 labexp4.l
25-01-2026  19:11               271 labexp5.l
25-01-2026  19:17               552 labexp6 c program.txt
25-01-2026  19:19               611 labexp7 c program.txt
25-01-2026  19:24             1,376 labexp8 c program.txt
25-01-2026  19:39               269 labexp9.l
27-01-2026  10:35            37,870 lex.yy.c
08-01-2026  14:56                74 path's.txt
              12 File(s)        105,800 bytes
               2 Dir(s)  201,001,701,376 bytes free

C:\compiler design>set path=C:\Program Files (x86)\GnuWin32\bin

C:\compiler design>flex labexp1.l

C:\compiler design>set path=

C:\compiler design>set path=C:\Program Files\CodeBlocks\MinGW\bin

C:\compiler design>gcc lex.yy.c

C:\compiler design>a.exe
Enter the operationa+b
Identifier
Operator
Identifier
```

Lab experiment 2 :

```
C:\Windows\System32\cmd.e

 Directory of C:\compiler design

28-01-2026  09:48    <DIR>          .
28-01-2026  09:48            63,547 a.exe
25-01-2026  18:37               311 labexp1.l
25-01-2026  18:43               288 labexp2.l
25-01-2026  18:59               302 labexp3.l
25-01-2026  19:07               329 labexp4.l
25-01-2026  19:11               271 labexp5.l
25-01-2026  19:17               552 labexp6 c program.txt
25-01-2026  19:19               611 labexp7 c program.txt
25-01-2026  19:24             1,376 labexp8 c program.txt
25-01-2026  19:39               269 labexp9.l
28-01-2026  09:48            37,714 lex.yy.c
08-01-2026  14:56                74 path's.txt
              12 File(s)        105,644 bytes
               1 Dir(s)  200,913,829,888 bytes free

C:\compiler design>set path=C:\Program Files (x86)\GnuWin32\bin

C:\compiler design>flex labexp2.l

C:\compiler design>set path=C:\Program Files\CodeBlocks\MinGW\bin

C:\compiler design>gcc lex.yy.c

C:\compiler design>a.exe
Enter a command : // this is a command
Single-line comment
```

Lab Experiment 3 :

```
C:\Windows\System32\cmd.e

28-01-2026  09:51    <DIR>          .
28-01-2026  09:51            63,547 a.exe
25-01-2026  18:37               311 labexp1.l
25-01-2026  18:43               288 labexp2.l
25-01-2026  18:59               302 labexp3.l
25-01-2026  19:07               329 labexp4.l
25-01-2026  19:11               271 labexp5.l
25-01-2026  19:17               552 labexp6 c program.txt
25-01-2026  19:19               611 labexp7 c program.txt
25-01-2026  19:24             1,376 labexp8 c program.txt
25-01-2026  19:39               269 labexp9.l
28-01-2026  09:51            37,714 lex.yy.c
08-01-2026  14:56                74 path's.txt
              12 File(s)        105,644 bytes
               1 Dir(s)  200,910,729,216 bytes free

C:\compiler design>set path=C:\Program Files (x86)\GnuWin32\bin

C:\compiler design>flex labexp3.l

C:\compiler design>set path=C:\Program Files\CodeBlocks\MinGW\bin

C:\compiler design>gcc lex.yy.c

C:\compiler design>a.exe
Enter an operator:
+
Valid Operator: +
```

Lab Experiment 4 :

```
C:\Windows\System32\cmd.e

25-01-2026  18:43               288 labexp2.l
25-01-2026  18:59               302 labexp3.l
25-01-2026  19:07               329 labexp4.l
25-01-2026  19:11               271 labexp5.l
25-01-2026  19:17               552 labexp6 c program.txt
25-01-2026  19:19               611 labexp7 c program.txt
25-01-2026  19:24             1,376 labexp8 c program.txt
25-01-2026  19:39               269 labexp9.l
28-01-2026  09:53            37,591 lex.yy.c
08-01-2026  14:56                74 path's.txt
              12 File(s)        105,521 bytes
               1 Dir(s)  200,910,364,672 bytes free

C:\compiler design>set path=C:\Program Files (x86)\GnuWin32\bin

C:\compiler design>flex labexp4.l

C:\compiler design>set path=C:\Program Files\CodeBlocks\MinGW\bin

C:\compiler design>gcc lex.yy.c

C:\compiler design>a.exe
Hello world
This is lex
^Z
Write something :
Number of whitespaces = 4
Number of new lines = 2
```

Lab Experiment 5 :

```
C:\Windows\System32\cmd.e

25-01-2026  19:11                271 labexp5.l
25-01-2026  19:17                552 labexp6 c program.txt
25-01-2026  19:19                611 labexp7 c program.txt
25-01-2026  19:24              1,376 labexp8 c program.txt
25-01-2026  19:39                269 labexp9.l
28-01-2026  09:54             37,521 lex.yy.c
08-01-2026  14:56                 74 path's.txt
               12 File(s)        105,487 bytes
                1 Dir(s)  200,909,398,016 bytes free

C:\compiler design>set path=C:\Program Files (x86)\GnuWin32\bin

C:\compiler design>flex labexp5.l

C:\compiler design>set path=C:\Program Files\CodeBlocks\MinGW\bin

C:\compiler design>gcc lex.yy.c

C:\compiler design>a.exe
Enter an identifier:
8
Invalid Identifier

+
Invalid Identifier

a
Valid Identifier
```

Lab experiment 6 :

```c
int main() {
    char nonTerminal;
    char alpha[20], beta[20];

    printf("Enter the non-terminal: ");
    scanf(" %c", &nonTerminal);

    printf("Enter alpha (left recursive part): ");
    scanf("%s", alpha);

    printf("Enter beta (non left recursive part): ");
    scanf("%s", beta);

    printf("\nGrammar without left recursion:\n");
    printf("%c -> %s%c'\n", nonTerminal, beta, nonTerminal);
    printf("%c' -> %s%c' | ε\n", nonTerminal, alpha, nonTerminal);

    return 0;
}
```

```
Enter the non-terminal: A
Enter alpha (left recursive part): a
Enter beta (non left recursive part): b

Grammar without left recursion:
A -> bA'
A' -> aA' | ε


=== Code Execution Successful ===
```

Lab experiment 7 :

main.c

```c

    printf("Enter the non-terminal: ");
    scanf(" %c", &nonTerminal);

    printf("Enter the common prefix: ");
    scanf("%s", common);

    printf("Enter first remaining part: ");
    scanf("%s", prod1);

    printf("Enter second remaining part: ");
    scanf("%s", prod2);

    printf("\nGrammar after left factoring:\n");
    printf("%c -> %s%c'\n", nonTerminal, common, nonTerminal);
    printf("%c' -> %s | %s\n", nonTerminal, prod1, prod2);

    return 0;
}

```

Output

```
Enter the non-terminal: A
Enter the common prefix: a
Enter first remaining part: b
Enter second remaining part: c

Grammar after left factoring:
A -> aA'
A' -> b | c


=== Code Execution Successful ===
```

Lab experiment 8 :

main.c

```c
#include <stdio.h>
#include <string.h>

struct symbol {
    char name[20];
    char type[20];
};

int main() {
    struct symbol table[10];
    int n = 0, choice, i;
    char key[20];

    while (1) {
        printf("\n1.Insert\n2.Search\n3.Display\n4.Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {

        case 1:
            printf("Enter symbol name: ");
            scanf("%s", table[n].name);
            printf("Enter symbol type: ");
            scanf("%s", table[n].type);
            n++;
            break;
```

Output

```
Enter symbol name: b
Enter symbol type: int

1.Insert
2.Search
3.Display
4.Exit
Enter your choice: 1
Enter symbol name: 2
Enter symbol type: int

1.Insert
2.Search
3.Display
4.Exit
Enter your choice: 3

Symbol Table:
Name    Type
a   int
b   int
2   int

1.Insert
2.Search
3.Display
4.Exit
Enter your choice:
```

Lab experiment 9 :

main.c

```c
printf("Enter Number of Production : ");
scanf("%d",&num);
printf("Enter the grammar as E->E-A :\n");
for(int i=0;i<num;i++){
scanf("%s",production[i]);
}
for(int i=0;i<num;i++){
printf("\nGRAMMAR : : : %s",production[i]);
non_terminal=production[i][0];
if(non_terminal==production[i][index]) {
alpha=production[i][index+1];
printf(" is left recursive.\n");
while(production[i][index]!=0 && production[i][index]!='|')
index++;
if(production[i][index]!=0) {
beta=production[i][index+1];
printf("Grammar without left recursion:\n");
printf("%c->%c%c\'",non_terminal,beta,non_terminal);
printf("\n%c\'->%c%c\'|E\n",non_terminal,alpha,non_terminal);
}
else
printf(" can't be reduced\n");
}
else
printf(" is not left recursive.\n");
index=3;
}
}
```

Output

```
Enter Number of Production : 2
Enter the grammar as E->E-A :
s-(l)|a
L->L,S|S


GRAMMAR : : : s-(l)|a is not left recursive.

GRAMMAR : : : L->L,S|S is left recursive.
Grammar without left recursion:
L->SL'
L'->,L'|E


=== Code Execution Successful ===
```

Lab experiment 10 :

main.c

```c
    // Create new production
    j = 0;
    for (i = pos; part1[i] != '\0'; i++)
        newGram[j++] = part1[i];

    newGram[j++] = '|';

    for (i = pos; part2[i] != '\0'; i++)
        newGram[j++] = part2[i];

    newGram[j] = '\0';

    // Output
    printf("\nAfter Left Factoring:");
    printf("\nS -> %s", modifiedGram);
    printf("\nX -> %s\n", newGram);

    return 0;
}
```

Output

```
Enter Production (example: ab|ac): S->iEtS|iEtSeS|a

After Left Factoring:
S -> X
X -> S->iEtS|iEtSeS|a


=== Code Execution Successful ===
```

Lab experiment 11 :

main.c

```c
printf("no such symbol");
else
{
printf("label is found \n");
printf("enter the address");
scanf("%d",&ad);
for(i=0;i<cnt;i++)
{
if(strcmp(sy[i].label,lab)==0)
sy[i].addr=ad;
}
}
}
void display()
{
int i;
for(i=0;i<cnt;i++)
printf("%s\t%d\n",sy[i].label,sy[i].addr);
}
```

Output

```
enter the address200

1.insert
2.display
3.search
4.modify
5.exit
2
a   100
b   200

1.insert
2.display
3.search
4.modify
5.exit
```

Lab experiment 12 :

main.c

```c
{
    if (input[i] == '(')
    {
        i++;
        if (E() && input[i] == ')')
        {
            i++;
            return 1;
        }
        return 0;
    }
    else if ((input[i] >= 'a' && input[i] <= 'z') ||
             (input[i] >= 'A' && input[i] <= 'Z'))
    {
        i++;
        return 1;
    }
    return 0;
}
```

Output

```
Recursive Descent Parsing for the Grammar:
E  -> T E'
E' -> + T E' | @
T  -> F T'
T' -> * F T' | @
F  -> (E) | id

Enter the string to be checked: (a+b)*c

String is ACCEPTED


=== Code Execution Successful ===
```

Lab experiment 13 :

```c
        {
            printf("String is NOT accepted\n");
            return 0;
        }
        else if (string[i] != 'a')
        {
            printf("String is NOT accepted\n");
            return 0;
        }
    }

    // Must contain at least one 'b'
    if (flag == 1)
        printf("String is ACCEPTED\n");
    else
        printf("String is NOT accepted\n");

    return 0;
}
```

Output

```
The grammar is: S -> aS | Sb | ab
Enter the string to be checked: abb
String is ACCEPTED


=== Code Execution Successful ===
```

Lab Experiment 14 :

```c
        stack[st_ptr - 1] == '*' ||
        stack[st_ptr - 1] == '/') &&
        stack[st_ptr - 2] == 'E')
    {
        stack[st_ptr - 2] = 'E';
        stack[st_ptr - 1] = '\0';
        st_ptr = st_ptr - 1;

        printf("\n$%s\t\t%s$\t\tREDUCE E->E%cE",
                stack, input, stack[st_ptr]);
    }

    // Accept conditiona
    if (strcmp(stack, "E") == 0 && ip_ptr == len)
    {
        printf("\n$%s\t\t%s$\t\tACCEPT", stack, input);
        exit(0);
    }
}
```

Output

```
    SHIFT REDUCE PARSER

GRAMMAR
E -> E+E | E*E | E/E | a | b

Enter the input string: a+b

STACK       INPUT       ACTION
$       a+b$        --
$a       +b$        SHIFT
$E       +b$        REDUCE E->E
$E+       b$        SHIFT
$E+b           $        SHIFT
$E+E           $        REDUCE E->E
$E         $        REDUCE E->E E
$E         $        ACCEPT

=== Code Execution Successful ===
```

Lab experiment 15 :

main.c

```c
                printf("\n");
                dispstack();
                printf("\t");
                dispinput();
                printf("\tREDUCE E->%s", lasthandle);
            }
            else
                break;
        }
    }

    if (strcmp(stack, "$E$") == 0)
        printf("\nAccepted\n");
    else
        printf("\nNot Accepted\n");

    free(input);
    return 0;
}
```

Output

```
Enter the string: i*(i+i)*i

STACK   INPUT   ACTION
$i  *(i+i)*i$   SHIFT
$E  *(i+i)*i$   REDUCE E->i
$E* (i+i)*i$    SHIFT
$E*(    i+i)*i$ SHIFT
$E*(i   +i)*i$  SHIFT
$E*(E   +i)*i$  REDUCE E->i
$E*(E+  i)*i$   SHIFT
$E*(E+i )*i$    SHIFT
$E*(E+E )*i$    REDUCE E->i
$E*(E+E)    *i$ SHIFT
$E*(E+E)*   i$  SHIFT
$E*(E+E)*i  $   SHIFT
$E*(E+E)*E  $   REDUCE E->i
$E*(E+E)*E$     SHIFT
Not Accepted
```

Lab experiment 16 :

main.c

```c
    char lhs[10], a[10], b[10], c[10];
    char op1, op2;

    printf("Enter expression (example: x = a + b - c):\n");
    scanf("%s = %s %c %s %c %s", lhs, a, &op1, b, &op2, c);

    printf("\nTHREE ADDRESS CODE\n");

    // First TAC
    printf("t1 = %s %c %s\n", a, op1, b);

    // Second TAC
    printf("t2 = t1 %c %s\n", op2, c);

    // Final assignment
    printf("%s = t2\n", lhs);

    return 0;
}
```

Output

```
Enter expression (example: x = a + b - c):
x=a-b
x=a+b-c

THREE ADDRESS CODE
t1 = ~
t2 = t1
x=a-b = t2


=== Code Execution Successful ===
```

Lab experiment 18 :

main.c

```c
        case '+':
            printf("\nADD %c, R%d", a[i][5], i);
            break;
        case '-':
            printf("\nSUB %c, R%d", a[i][5], i);
            break;
        case '*':
            printf("\nMUL %c, R%d", a[i][5], i);
            break;
        case '/':
            printf("\nDIV %c, R%d", a[i][5], i);
            break;
        }

        printf("\nMOV R%d, %c\n", i, a[i][0]);
    }

    return 0;
}
```

Output

```
Enter number of three address codes: 2

Enter the three address code (example: x=a+b)
a=b*c
x=a+b

Generated Target Code:

MOV *, R0
MOV R0, a

MOV +, R1
MOV R1, x


=== Code Execution Successful ===
```

Lab experiment 19 :

main.c

```c
    printf("\nFIRST Sets\n");

    for (i = 0; i < 18; i++)
    {
        if (prev != arr[i][0])
        {
            prev = arr[i][0];
            printf("\nFIRST(%c) = { ", prev);
        }

        if (arr[i][2] == 'T')
            printf("%c ", arr[i][1]);

        if (i == 17 || arr[i + 1][0] != prev)
            printf("}");
    }

    return 0;
}
```

Output

```
  F     +   F
  F     *   F
  F     (   T
  F     )   F
  F     i   T
  F     $   F
  T     +   F
  T     *   T
  T     (   T
  T     )   F
  T     i   T
  T     $   F

FIRST Sets

FIRST(E) = { + * ( i }
FIRST(F) = { ( i }
FIRST(T) = { * ( i }

=== Code Execution Successful ===
```

Lab experiment 20 :

```c
    /* Display FIRST sets */
    printf("\nFIRST SETS\n");
    for (i = 0; i < 18; i++)
    {
        if (prev != arr[i][0])
        {
            prev = arr[i][0];
            printf("\nFIRST(%c) = { ", prev);
        }
        if (arr[i][2] == 'T')
            printf("%c ", arr[i][1]);

        if (i == 17 || arr[i + 1][0] != prev)
            printf("}");
    }

    return 0;
}
```

```
F   +   F
F   *   F
F   (   T
F   )   F
F   i   T
F   $   F
T   +   F
T   *   T
T   (   T
T   )   F
T   i   T
T   $   F

FIRST SETS

FIRST(E) = { + * ( i }
FIRST(F) = { ( i }
FIRST(T) = { * ( i }
```

Lab experiment 21 :

```
28-01-2026  10:42               381 labexp21.l
25-01-2026  18:59               302 labexp3.l
25-01-2026  19:07               329 labexp4.l
25-01-2026  19:11               271 labexp5.l
25-01-2026  19:17               552 labexp6 c program.txt
25-01-2026  19:19               611 labexp7 c program.txt
25-01-2026  19:24             1,376 labexp8 c program.txt
28-01-2026  10:14             1,006 labexp9.l
28-01-2026  10:09            38,229 lex.yy.c
08-01-2026  14:56                74 path's.txt
               18 File(s)        115,382 bytes
                1 Dir(s)  200,809,840,640 bytes free

C:\compiler design>set path=C:\Program Files (x86)\GnuWin32\bin

C:\compiler design>flex labexp21.l

C:\compiler design>set path=C:\Program Files\CodeBlocks\MinGW\bin

C:\compiler design>gcc lex.yy.c

C:\compiler design>a.exe
G:\lex>flex count_line.l
233
^Z
Number of characters = 29
Number of words = 3
Number of lines = 2

C:\compiler design>
```

Lab exp 22 :